



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه ششم – تقسیم و حل: مرتب‌سازی ادغامی و تحلیل الگوریتم‌های بازگشتی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

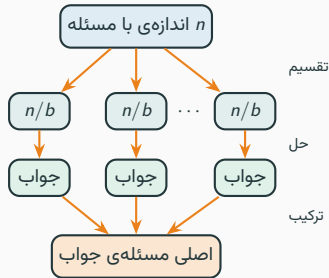
در پایان این جلسه انتظار داریم که:

- ایده‌ی **مرتب‌سازی ادغامی** را بشناسیم و اجرای آن را روی یک آرایه دنبال کنیم.
- رویه‌ی MERGE را خطبه‌خط بفهمیم و ببینیم چرا زمانش $\Theta(n)$ است.
- برای یک الگوریتم تقسیم و حل **رابطه‌ی بازگشتی** بنویسیم.
- با **درخت بازگشت** نشان دهیم زمان مرتب‌سازی ادغامی $\Theta(n \lg n)$ است.

یادآوری: روش تقسیم و حل

«بسیاری از الگوریتم‌های سودمند ساختاری **بازگشتی** دارند: برای حل یک مسئله‌ی داده‌شده، یک یا چند بار **بازگشت می‌کنند** (خودشان را صدا می‌زنند) تا زیرمسئله‌هایی را که ارتباط نزدیکی با آن دارند حل کنند. این الگوریتم‌ها معمولاً از روش **تقسیم و حل** پیروی می‌کنند: مسئله را به چند زیرمسئله می‌شکنند که شبیه مسئله‌ی اصلی‌اند اما اندازه‌ی کوچک‌تری دارند، زیرمسئله‌ها را به‌صورت بازگشتی حل می‌کنند و سپس این جواب‌ها را ترکیب می‌کنند تا جوابی برای مسئله‌ی اصلی بسازند.»

CLRS, 4th ed., صفحه ۳۴ (ترجمه ماشینی)



«در روش تقسیم و حل، اگر مسئله به اندازه‌ی کافی کوچک باشد (حالت پایه)، آن را مستقیماً و بدون بازگشت حل می‌کنید. در غیر این صورت (حالت بازگشتی)، سه گام مشخص را انجام می‌دهید:

- **تقسیم** مسئله به یک یا چند زیرمسئله که نمونه‌های کوچک‌تری از همان مسئله‌اند.
- **حل** زیرمسئله‌ها با حل بازگشتی آن‌ها.
- **ترکیب** جواب‌های زیرمسئله‌ها برای ساختن جوابی برای مسئله‌ی اصلی.»

CLRS, 4th ed., صفحه ۳۴ (ترجمه ماشینی).

ایده‌ی مرتب‌سازی ادغامی

برای مرتب کردن زیرآرایه‌ی $A[p : r]$ (از کل آرایه‌ی $A[1 : n]$ شروع می‌کنیم):

- **تقسیم:** نقطه‌ی میانی q را حساب کن و $A[p : r]$ را به دو نیمه‌ی $A[p : q]$ و $A[q + 1 : r]$ بشکن.
- **حل:** هر نیمه را با یک فراخوانی بازگشتی مرتب کن.
- **ترکیب:** دو نیمه‌ی مرتب را با MERGE در هم ادغام کن تا $A[p : r]$ مرتب شود.

«عمل کلیدی الگوریتم مرتب‌سازی ادغامی در گام “ترکیب” رخ می‌دهد که دو زیرآرایه‌ی مجاور و مرتب را ادغام می‌کند.»

CLRS, 4th ed., صفحه ۳۵ (ترجمه ماشینی)

- **حالت پایه:** وقتی $p = r$ باشد، زیرآرایه فقط یک عنصر دارد و خودبه‌خود مرتب است؛ پس بازگشت همین‌جا متوقف می‌شود.
- **رویه $\text{MERGE}(A, p, q, r)$:** با شرط $p \leq q < r$ ، فرض می‌کند $A[p : q]$ و $A[q + 1 : r]$ هر دو مرتب‌اند و از آن‌ها یک زیرآرایه‌ی مرتب در $A[p : r]$ می‌سازد.

رویهی MERGE

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                      //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                      //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                      //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element, copy the smallest unmerged element back into  $A[p : r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
18          $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the remainder of the other to the end of  $A[p : r]$ .
20 while  $i < n_L$ 
21      $A[k] = L[i]$ 
22      $i = i + 1$ 
23      $k = k + 1$ 
24 while  $j < n_R$ 
25      $A[k] = R[j]$ 
26      $j = j + 1$ 
27      $k = k + 1$ 

```

۳۶ صفحه، CLRS، 4th ed.

زیرآرایه‌ی $\langle 2, 4, 6, 7, 1, 2, 3, 5 \rangle$ $A[9 : 16]$ است و هر دو نیمه‌ی آن مرتب‌اند. پس از خط‌های ۱ تا ۷:
 $R = \langle 1, 2, 3, 5 \rangle$ و $L = \langle 2, 4, 6, 7 \rangle$

راهنمای رنگ‌ها در ردیابی:

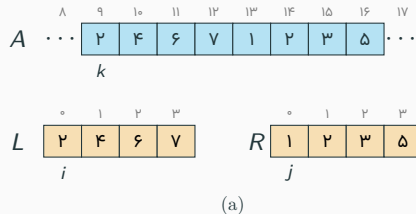
- **کرم** در A : مقدار نهایی در جای خودش نشسته است.
- **آبی** در A : مقدار قدیمی که هنوز رونویسی می‌شود.
- **کرم** در L و R : هنوز به A برگشته است.
- **تیره** در L و R : قبلاً به A برگشته است.

مثال و رنگ‌بندی از *CLRS, 4th ed.*، شکل ۳.۲، صفحه ۳۷

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
19   :
20   :
21 while  $i < n_L$ 
22    $A[k] = L[i]$ 
23    $i = i + 1$ 
24    $k = k + 1$ 
25 while  $j < n_R$ 
26    $A[k] = R[j]$ 
27    $j = j + 1$ 
28    $k = k + 1$ 

```



تیره آبی کرم

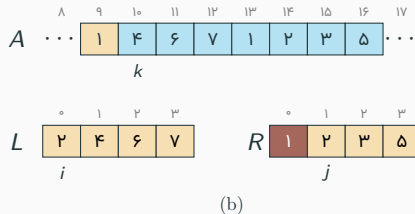
پیش از تکرار ۱: $L[0] = 2$ و $R[0] = 1$ مقایسه می‌شوند. چون $2 \leq 1$ برقرار نیست، else اجرا می‌شود: $A[9] = 1$ و j یک واحد جلو می‌رود.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



تیره آبی کرم

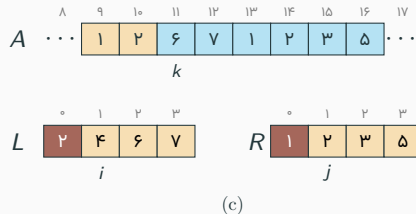
پیش از تکرار ۲: $L[0] = 2$ و $R[1] = 2$. شرط $2 \leq 2$ برقرار است، پس در حالت برابر از L برداشته می‌شود: $A[10] = 2$ و i جلو می‌رود.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
19   :
20   :
21 while  $i < n_L$ 
22    $A[k] = L[i]$ 
23    $i = i + 1$ 
24    $k = k + 1$ 
25 while  $j < n_R$ 
26    $A[k] = R[j]$ 
27    $j = j + 1$ 
28    $k = k + 1$ 

```



تیره آبی کرم

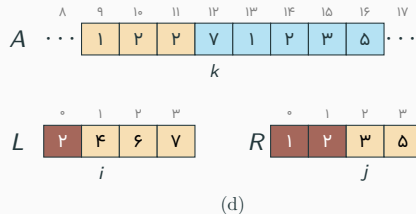
پیش از تکرار ۳: $L[1] = 4$ و $R[1] = 2$. عنصر R کوچک‌تر است: $A[11] = 2$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
  :
  :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



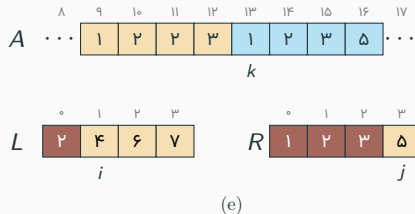
پیش از تکرار ۴: $L[1] = 4$ و $R[2] = 3$. باز هم از R : $A[12] = 3$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



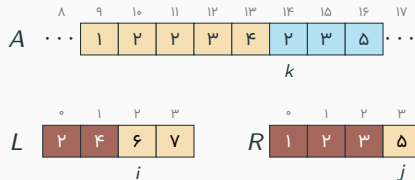
تیره آبی کرم

پیش از تکرار ۵: $L[1] = ۴$ و $R[۳] = ۵$. این بار از L : $A[۱۳] = ۴$.

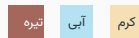
(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
    :
    :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 
    
```



(f)



پیش از تکرار ۶: $L[2] = 6$ و $R[3] = 5$ از $A[14] = 5$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

12 while $i < n_L$ and $j < n_R$

13 if $L[i] \leq R[j]$

14 $A[k] = L[i]$

15 $i = i + 1$

16 else $A[k] = R[j]$

17 $j = j + 1$

18 $k = k + 1$

⋮

20 while $i < n_L$

21 $A[k] = L[i]$

22 $i = i + 1$

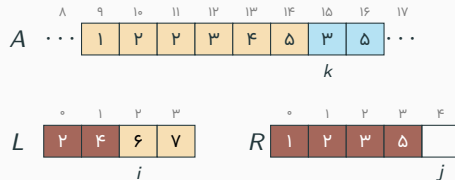
23 $k = k + 1$

24 while $j < n_R$

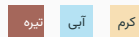
25 $A[k] = R[j]$

26 $j = j + 1$

27 $k = k + 1$



(g)



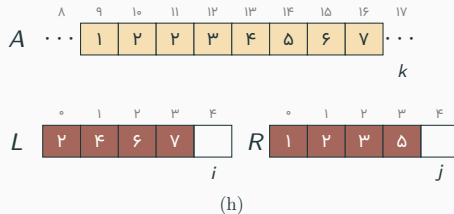
حالت (g): $n_R = 4$ ؛ همه‌ی R به A برگشته و حلقه‌ی خط‌های ۱۲-۱۸ تمام می‌شود. حلقه‌ی خط‌های ۲۰-۲۳ مقدارهای $L[2 : 3]$ یعنی ۶ و ۷ را در $A[15 : 16]$ کپی می‌کند.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



تیره آبی کرم

حالت (h): پایان. حلقه‌ی خط‌های ۲۴-۲۷ صفر بار اجرا شد و زیرآرایه‌ی $A[9 : 16]$ مرتب است.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

«...» هر تکرار این حلقه‌ها دقیقاً یک مقدار را از L یا R به A برمی‌گرداند و هر مقدار دقیقاً یک بار به A برگردانده می‌شود.»

CLRS, 4th ed.، صفحه ۳۸ (ترجمه ماشینی)

$$\underbrace{\Theta(1)}_{\text{خط‌های ۱-۳ و ۸-۱۰}} + \underbrace{\Theta(n_L + n_R)}_{\text{حلقه‌های for}} + \underbrace{\Theta(n)}_{\text{سه حلقه‌ی while}} = \Theta(n)$$

رویهی MERGE-SORT

```

MERGE-SORT( $A, p, r$ )
1  if  $p \geq r$                       // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$         // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )              // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )          // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
    
```

صفحه ۳۹، *CLRS*, 4th ed.

تقسیم: خط ۳ حل: خط‌های ۴ و ۵ ترکیب: خط ۷

۱	۲	۳	۴	۵	۶	۷	۸
۱۲	۳	۷	۹	۱۴	۶	۱۱	۲

تقسیم

تقسیم

تقسیم

جاری فراخوانی

MERGE-SORT($A, 1, 8$)

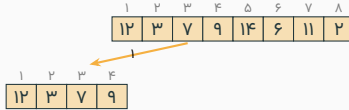
ادغام

ادغام

ادغام

CLRS, 4th ed.، صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)

تقسیم



تقسیم

تقسیم

جاری فراخوانی

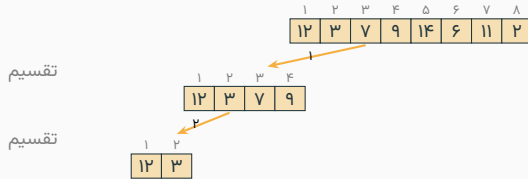
MERGE-SORT($A, 1, 4$)

ادغام

ادغام

ادغام

(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

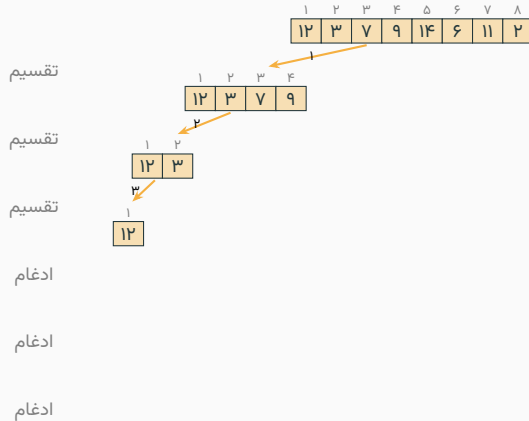
MERGE-SORT($A, 1, 2$)

ادغام

ادغام

ادغام

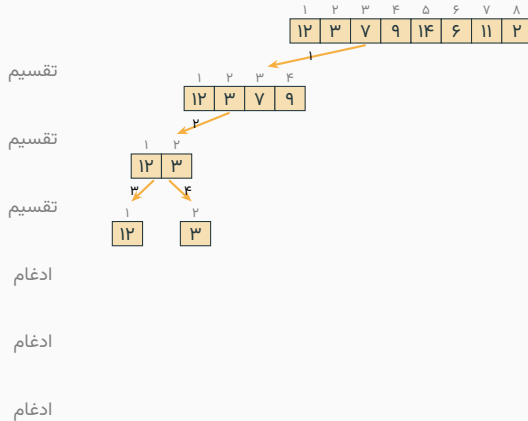
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT($A, 1, 1$)

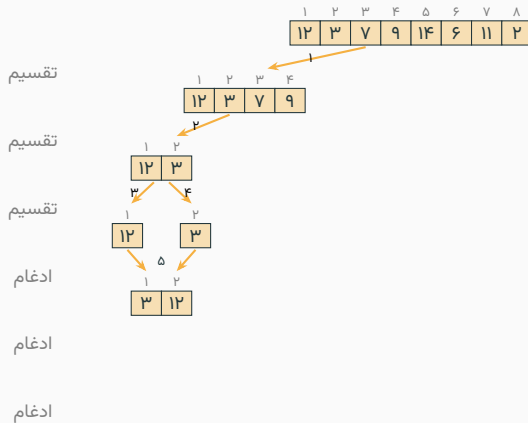
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p و q و r حذف شده‌اند)



جاری فراخوانی

MERGE-SORT($A, 2, 2$)

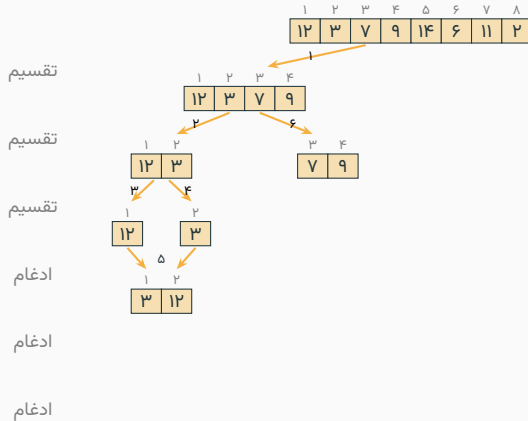
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE($A, 1, 1, 2$)

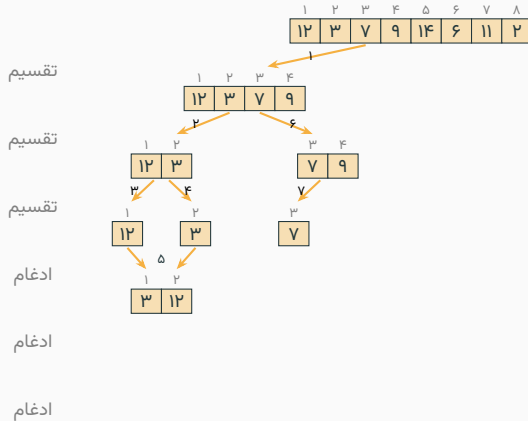
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 3, 4$)

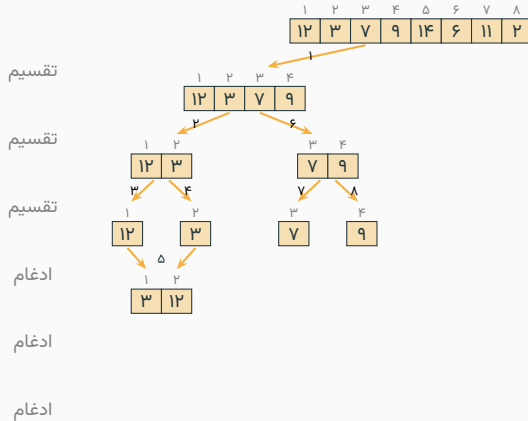
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 3, 3$)

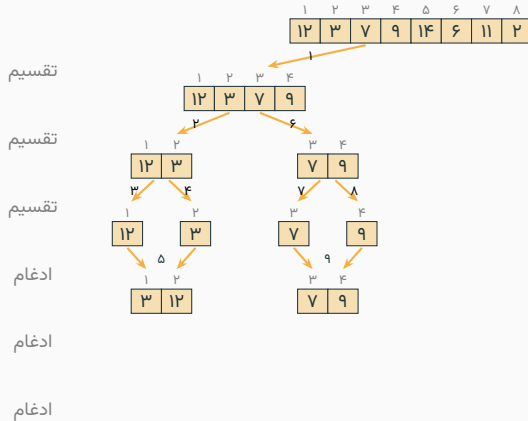
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE-SORT($A, 4, 4$)

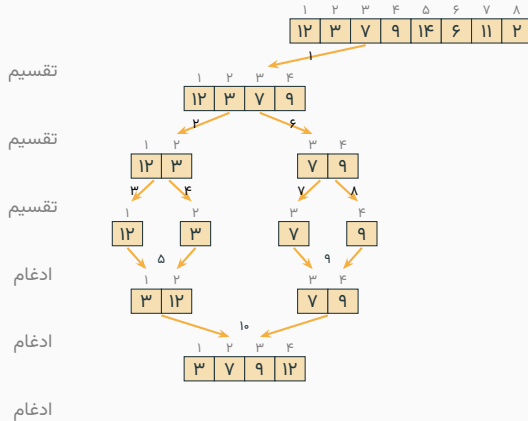
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE(A, 3, 3, 4)

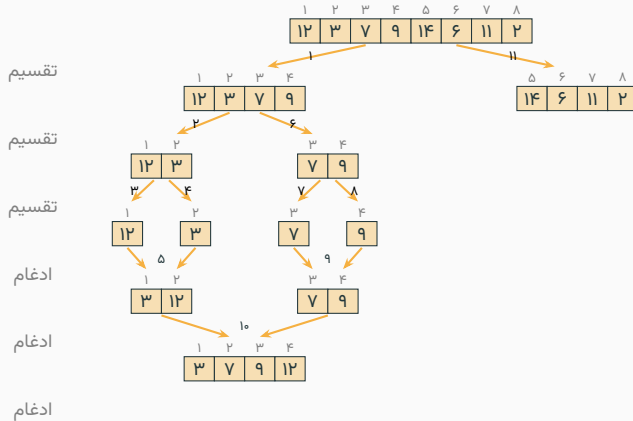
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE(A, 1, 2, 4)

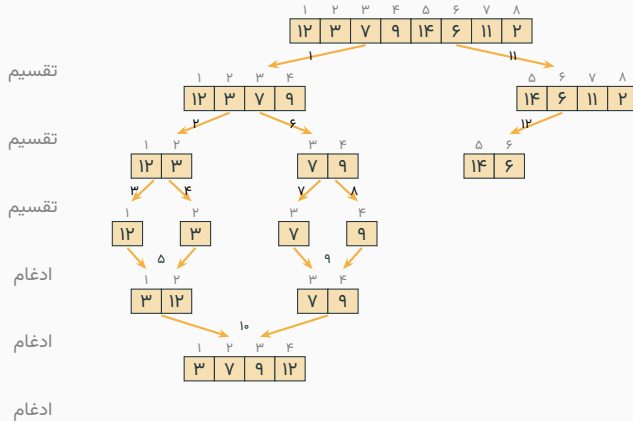
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 5, 8$)

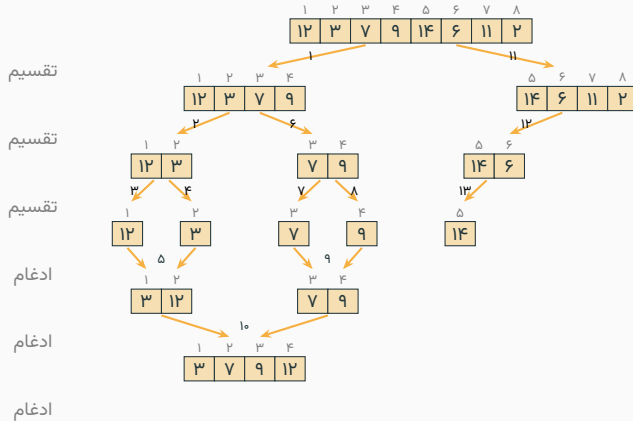
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT($A, 5, 6$)

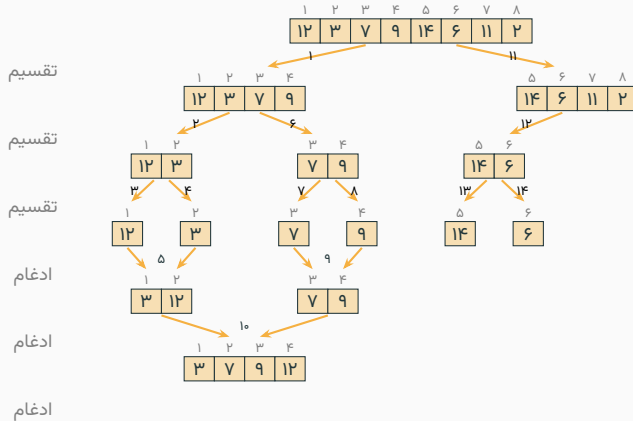
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT($A, 5, 5$)

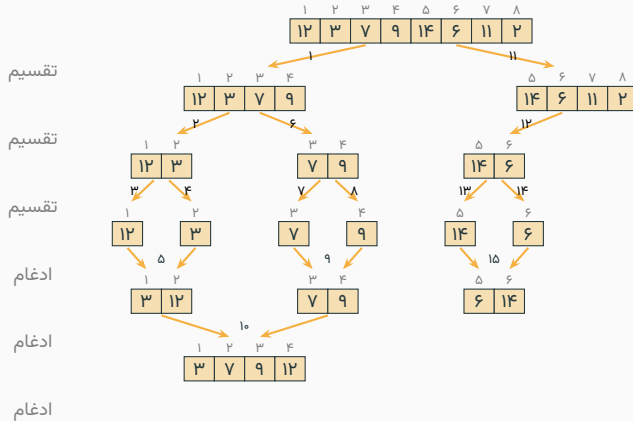
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT(A, 6, 6)

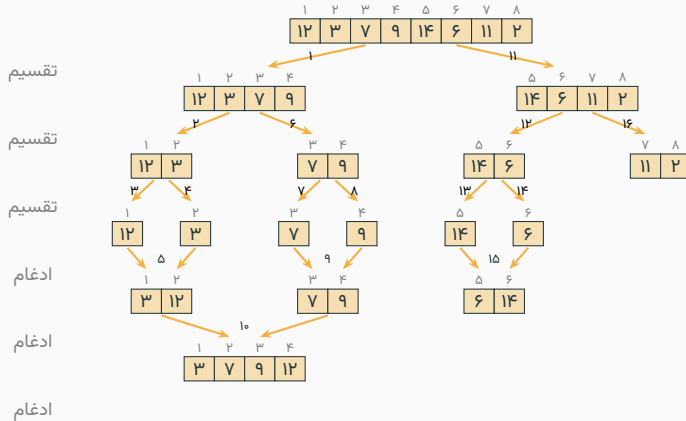
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 5, 5, 6)

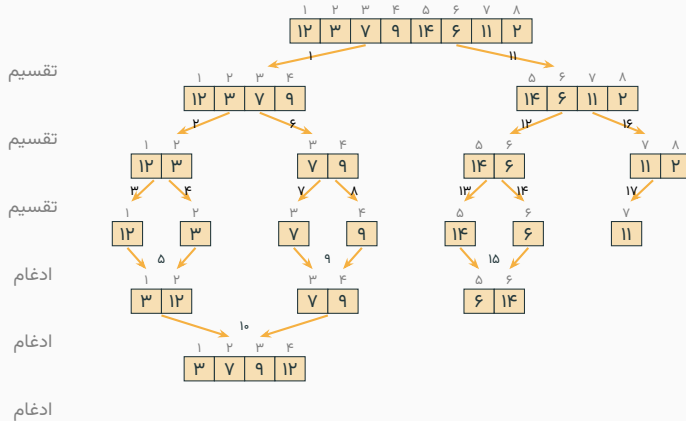
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT(A, 7, 8)

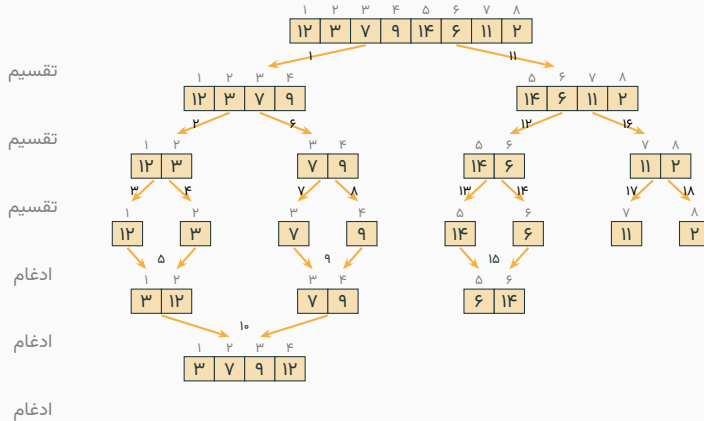
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



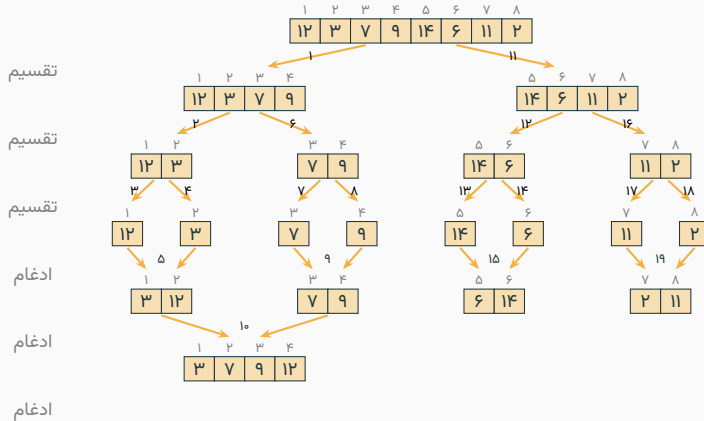
جاری فراخوانی

MERGE-SORT(A, 7, 7)

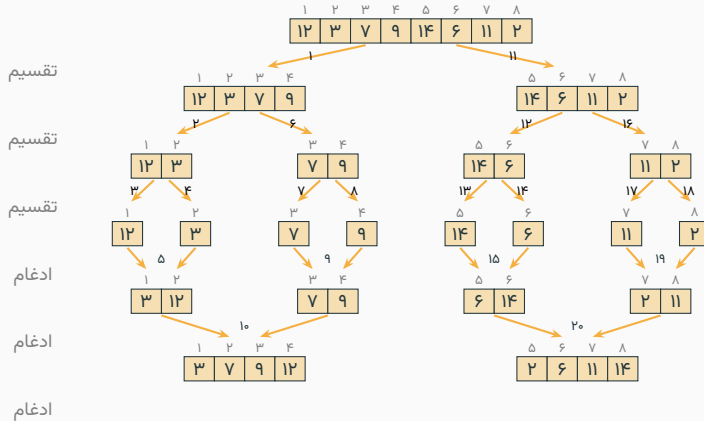
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



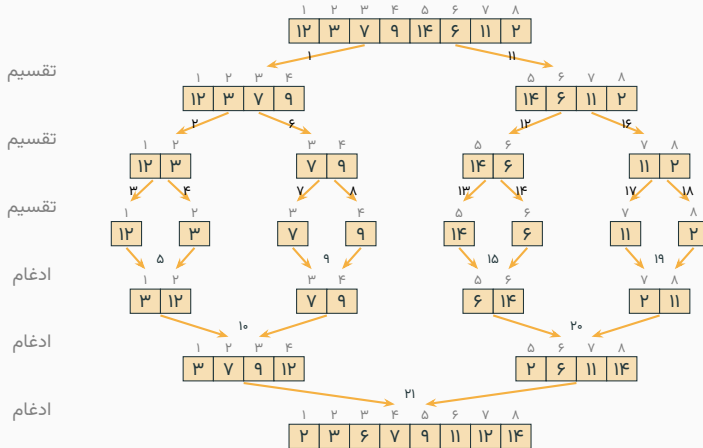
(بازترسیم شکل ۴.۲؛ برچسب‌های p , q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 5, 6, 8)

(بازترسیم شکل ۴.۲؛ برچسب‌های p , q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 1, 4, 8)

(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*

رابطه‌ی بازگشتی

«وقتی الگوریتمی یک فراخوانی بازگشتی دارد، اغلب می‌توانید زمان اجرایش را با یک **معادله‌ی بازگشتی** یا **رابطه‌ی بازگشتی** توصیف کنید، که زمان اجرای کل روی مسئله‌ای به اندازه‌ی n را برحسب زمان اجرای همان الگوریتم روی ورودی‌های کوچک‌تر توصیف می‌کند. سپس می‌توانید از ابزارهای ریاضی برای حل رابطه‌ی بازگشتی و ارائه‌ی کران‌هایی برای کارایی الگوریتم استفاده کنید.»

CLRS, 4th ed.، صفحه ۳۹ (ترجمه ماشینی)

$$T(n) = \begin{cases} \Theta(1), & n < n_0 \\ D(n) + aT(n/b) + C(n), & \text{در غیر این صورت} \end{cases}$$

بر اساس *CLRS, 4th ed.*، صفحه ۴۰

گام	خط در MERGE-SORT
$D(n) = \Theta(1)$	خط ۳
$2T(n/2)$	خط‌های ۴ و ۵
$C(n) = \Theta(n)$	خط ۷

«در این جا می‌بینید که رابطه‌ی بازگشتی برای $T(n)$ ، زمان اجرای بدترین حالت مرتب‌سازی ادغامی روی n عدد، چگونه نوشته می‌شود.

- **تقسیم:** گام تقسیم فقط وسط زیرآرایه را حساب می‌کند که زمان ثابتی می‌گیرد. بنابراین $D(n) = \Theta(1)$.
- **حل:** حل بازگشتی دو زیرمسئله، هر کدام به اندازه‌ی $n/2$ ، مقدار $2T(n/2)$ به زمان اجرا اضافه می‌کند (...).
- **ترکیب:** چون رویه‌ی MERGE روی یک زیرآرایه‌ی n عنصری زمان $\Theta(n)$ می‌گیرد، داریم $C(n) = \Theta(n)$ ».

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

«افزودن $\Theta(n)$ به جمله‌ی $۲T(n/۲)$ از گام حل، رابطه‌ی بازگشتی زمان اجرای بدترین حالت $T(n)$ مرتب‌سازی ادغامی را به دست می‌دهد:»

$$T(n) = ۲T(n/۲) + \Theta(n) \quad (۲,۳)$$

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

«فصل ۴ "قضیه‌ی اصلی" را ارائه می‌کند که نشان می‌دهد $T(n) = \Theta(n \lg n)$ »

$$T(n) = 2T(n/2) + \Theta(n) \implies T(n) = \Theta(n \lg n)$$

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

درخت بازگشت

«برای سادگی، فرض کنید n توانی دقیق از ۲ است و حالت پایه‌ی ضمنی $n = 1$ است. آنگاه رابطه‌ی بازگشتی (۳.۲) در اصل چنین است:»

$$T(n) = \begin{cases} c_1, & n = 1 \\ 2T(n/2) + c_2n, & n > 1 \end{cases} \quad (2.4)$$

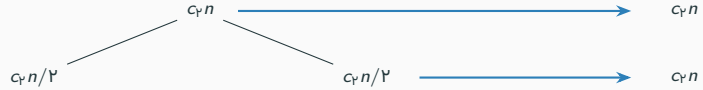
CLRS, 4th ed.، صفحه ۴۲ (ترجمه ماشینی).

$O(n)$

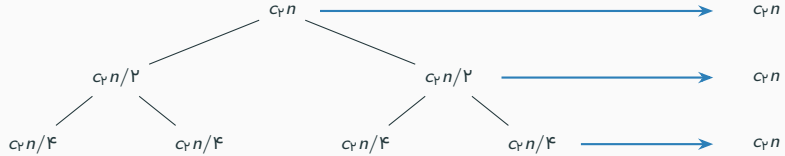
d) بخش ۵.۲، شکل ۵.۲، بخش ۴۳، *CLRS*, 4th ed.

$$c_2 n \longrightarrow c_2 n$$

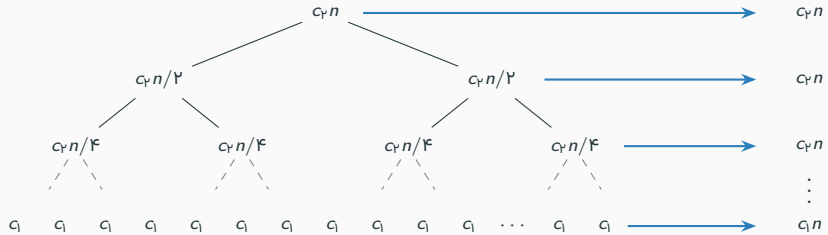
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش d)، *CLRS*, 4th ed.



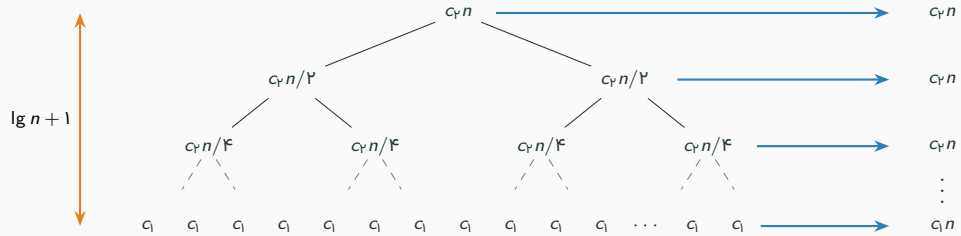
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲، *CLRS*, 4th ed.



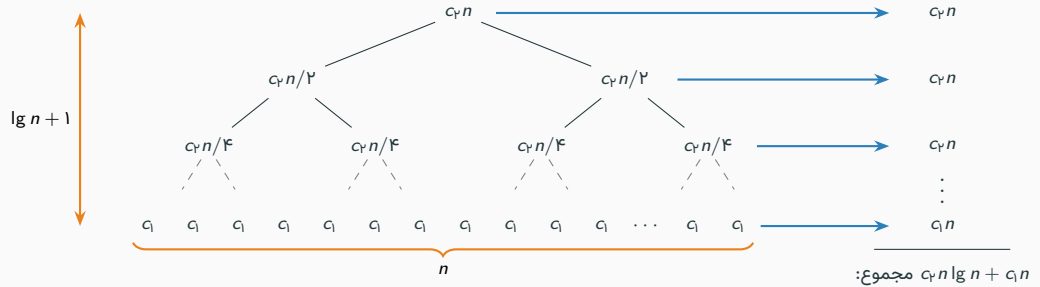
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.

«برای محاسبه‌ی هزینه‌ی کلی که رابطه‌ی بازگشتی (۴.۲) نشان می‌دهد، کافی است هزینه‌های همه‌ی سطح‌ها را جمع کنید. درخت بازگشت $\lg n + 1$ سطح دارد. هزینه‌ی هر یک از سطح‌های بالای برگ‌ها $c_2 n$ است و هزینه‌ی سطح برگ‌ها $c_1 n$ ، که هزینه‌ی کل آن $c_2 n \lg n + c_1 n = \Theta(n \lg n)$ است.»

CLRS, 4th ed.، صفحه ۴۴ (ترجمه ماشینی)

$$T(n) = \underbrace{c_2 n + c_2 n + \dots + c_2 n}_{\lg n} + \underbrace{c_1 n}_{\text{برگ‌ها}} = c_2 n \lg n + c_1 n = \Theta(n \lg n)$$

منابع

Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022 •